MOBILE DEVELOPMENT: IOS

Mark Joselli

mark.joselli@pucpr.br

ECOSSISTEMA APPLE

- O ecossistema Apple é um dos principais diferenciais da empresa e influencia diretamente o desenvolvimento de aplicativos para iOS.
- Ele é composto por um conjunto de dispositivos, sistemas operacionais e serviços que funcionam de maneira integrada, oferecendo uma experiência contínua para os usuários.
- iPhone, iPad, Mac, Apple Watch, Apple TV e Apple Vision Pro

VANTAGENS DE SE TORNAR UM DESENVOLVEDOR IOS

- Salários Altos 💰
  - Profissionais iOS costumam receber salários competitivos, muitas vezes superiores aos de outras áreas de desenvolvimento.
  - Nos Estados Unidos, o salário médio de um desenvolvedor iOS pode ultrapassar US$ 100.000 por ano.
  - No Brasil e em outros países, os salários variam, mas geralmente estão entre os mais altos do mercado de TI.
- Ecossistema Apple: Usuários Premium 🍏
  - O público da Apple tende a gastar mais dinheiro com aplicativos e assinaturas do que os usuários de Android.
  - Desenvolver para um ecossistema fechado como o da Apple garante experiência consistente entre dispositivos.
  - Maior controle de qualidade e segurança para os aplicativos, reduzindo fragmentação e compatibilidade.
- Possibilidade de Empreender 🚀
  - Qualquer desenvolvedor pode publicar seu próprio app na App Store e monetizá-lo.
  - Modelos de negócios incluem compras dentro do app (IAP), assinaturas, apps pagos ou até mesmo publicidade.
  - Histórias de sucesso como o Flappy Bird, que foi criado por um único desenvolvedor e gerava US$ 50 mil por dia, inspiram novos criadores.

VANTAGENS DE SE TORNAR UM DESENVOLVEDOR IOS

- Mercado de Trabalho Aquecido 🔥
  - A demanda por desenvolvedores iOS continua alta, já que aplicativos móveis são essenciais para empresas e startups.
  - Empresas de tecnologia, fintechs, healthtechs e e-commerce sempre precisam de profissionais para criar e manter apps nativos.
  - A quantidade de desenvolvedores Android é maior do que iOS, o que torna a concorrência menor no mercado Apple.
- Exclusividade e Diferenciação 📱
  - Muitos desenvolvedores iniciam em Android, mas poucos dominam iOS, tornando-se mais valiosos no mercado.
  - Empresas que desenvolvem apps para iOS geralmente priorizam qualidade, design e experiência do usuário.
- Trabalhar Remotamente ou Internacionalmente 🌍
  - Empresas globais contratam desenvolvedores iOS para projetos remotos.
  - Com experiência, é possível conseguir vagas internacionais e até trabalhar como freelancer para clientes ao redor do mundo.

Works like magic

No stylus

Far more accurate

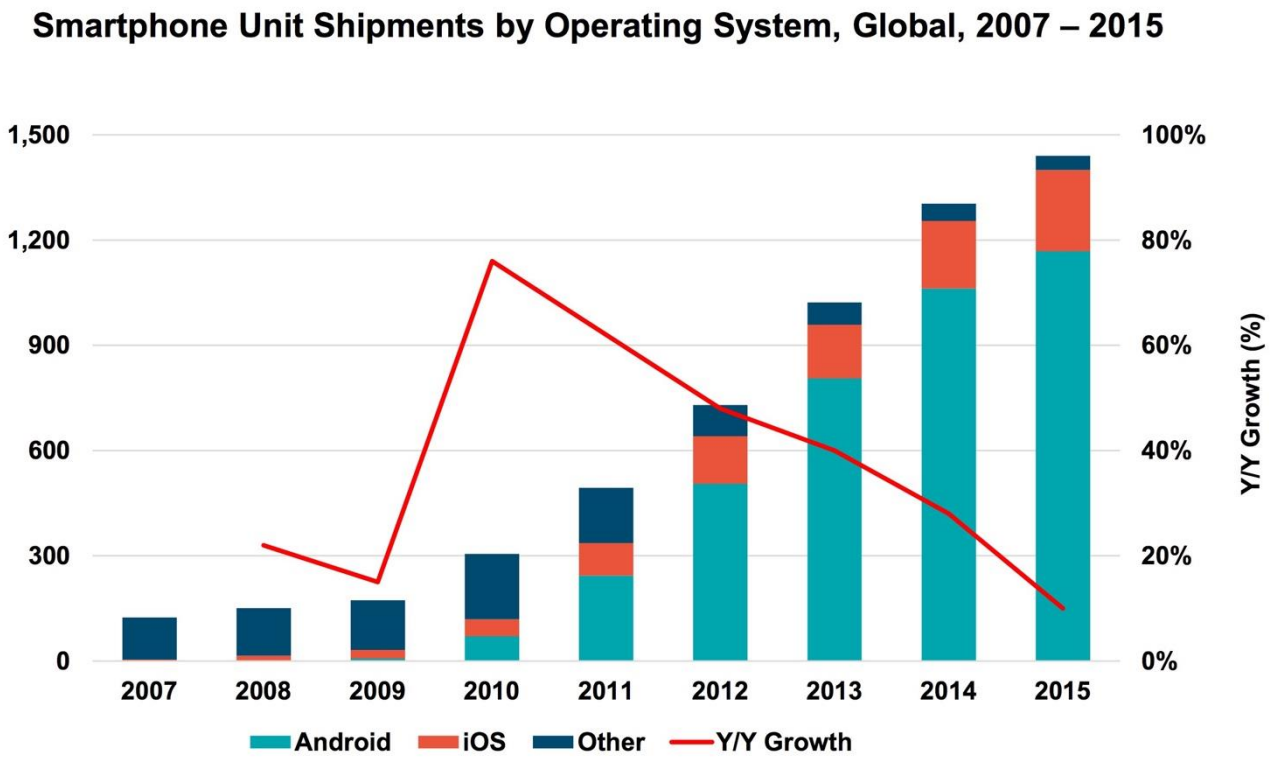
Ignores unintended touches

Multi-finger gestures

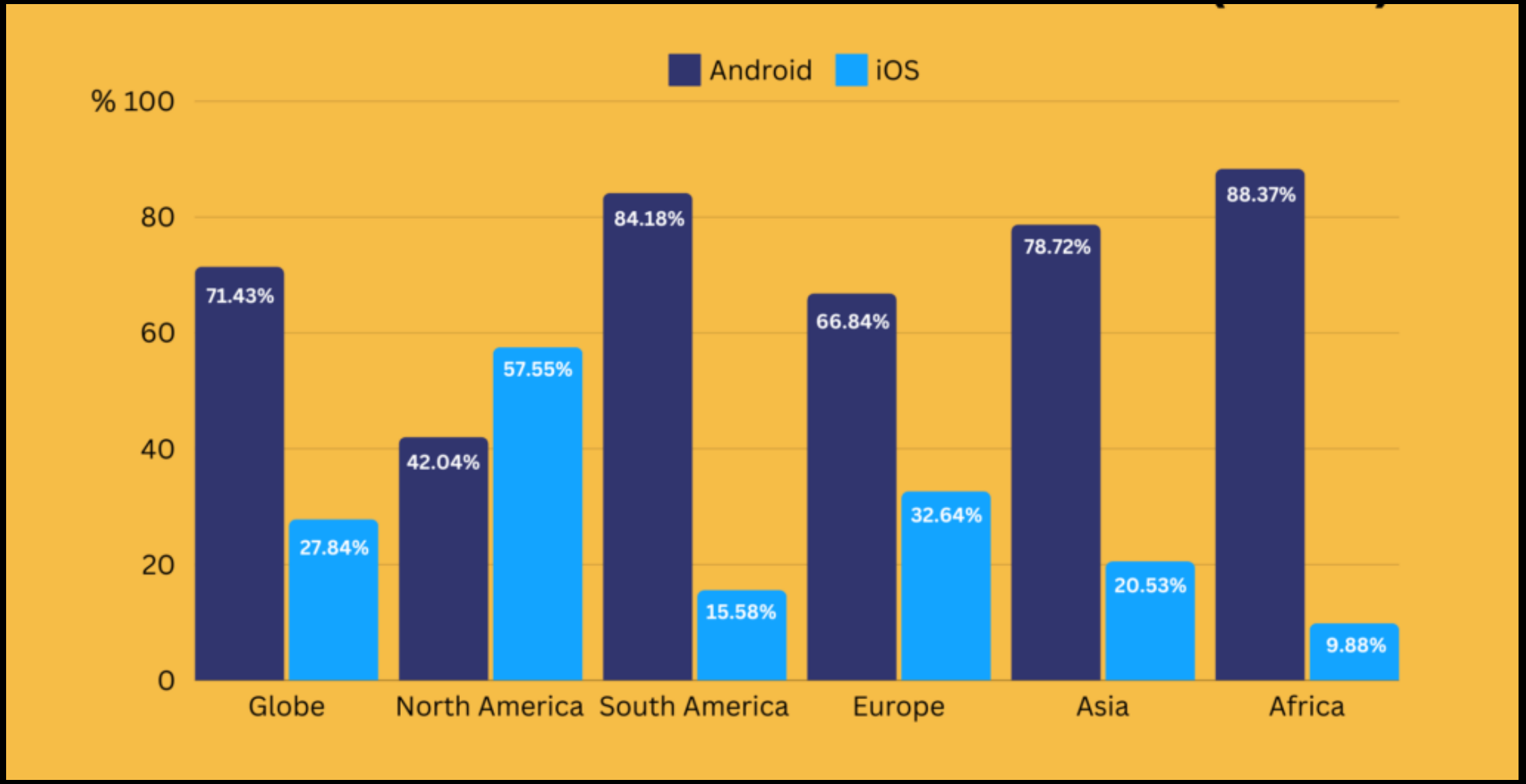
Patented !

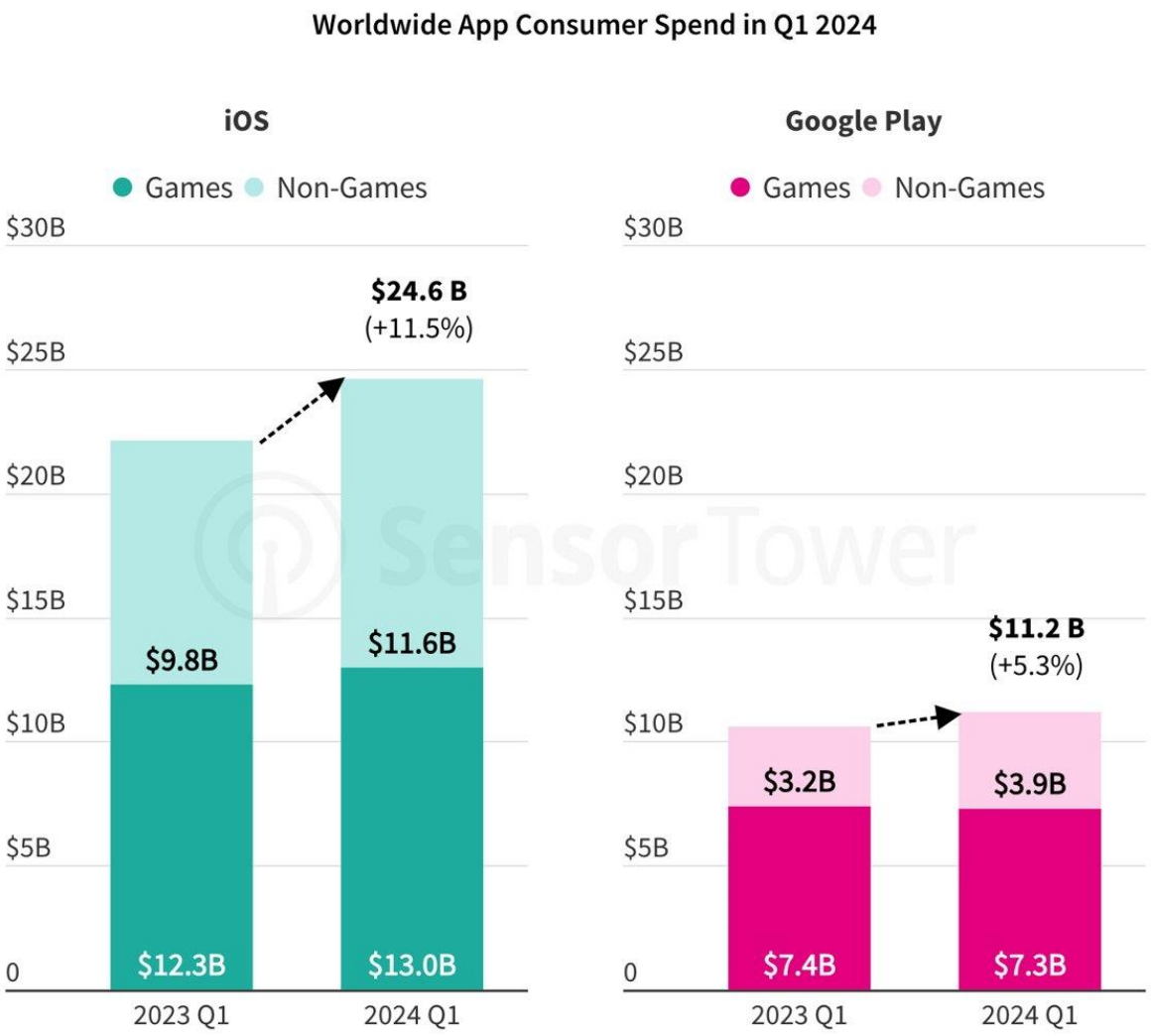
ANDROID X IOS

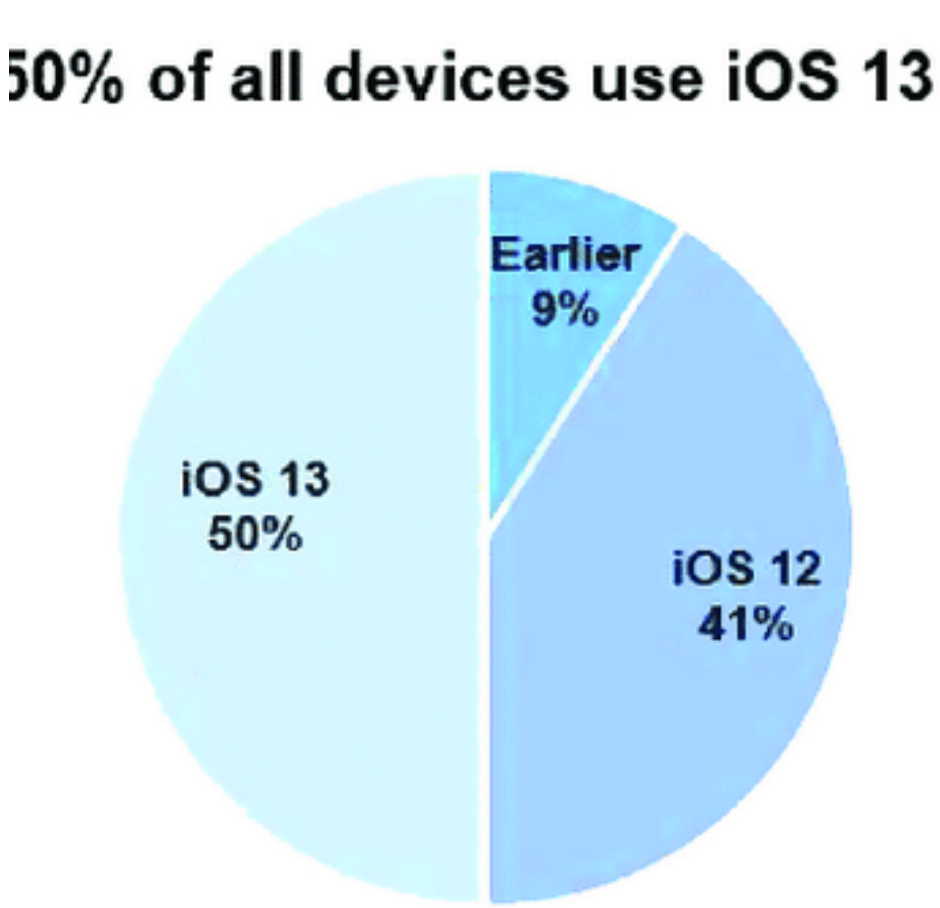
al Smartphone Units Slowing Dramatically...
5 Years of High Growth @ +10% vs. +28% Y/Y

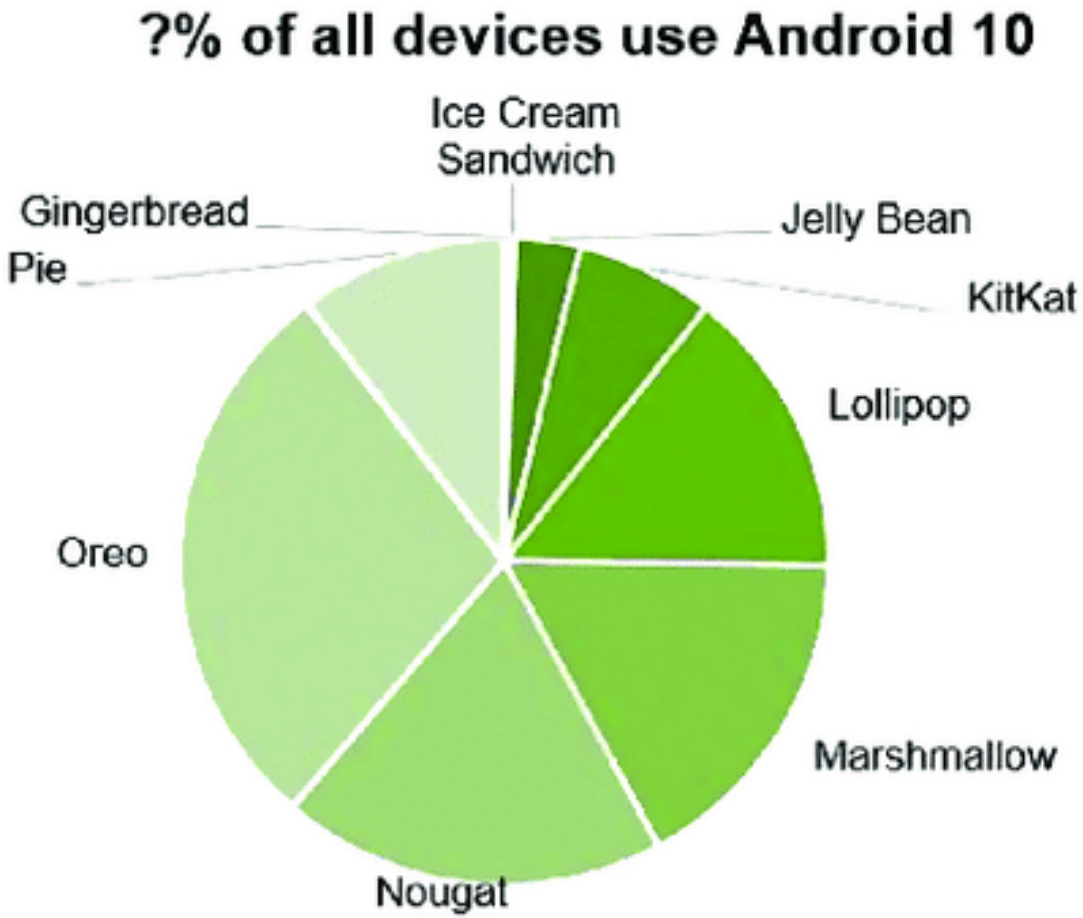


Source: Morgan Stanley Research, 5/16.


Android
iOS
% 100
80
60
40
20
0
71.43%
27.84%
42.04%
57.55%
84.18%
15.58%
66.84%
32.64%
78.72%
20.53%
88.37%
9.88%
Globe
North America
South America
Europe
Asia
Africa


Worldwide App Consumer Spend in Q1 2024
iOS
Games
Non-Games
$30B
$25B
$20B
$15B
$10B
$5B
0
$24.6 B
(+11.5%)
$9.8B
$11.6B
$12.3B
$13.0B
2023 Q1
2024 Q1
Google Play
Games
Non-Games
$30B
$25B
$20B
$15B
$10B
$5B
0
$11.2 B
(+5.3%)
$3.2B
$3.9B
$7.4B
$7.3B
2023 Q1
2024 Q1


50% of all devices use iOS 13
Earlier
9%
iOS 13
50%
iOS 12
41%


?% of all devices use Android 10
Ice Cream
Sandwich
Gingerbread
Jelly Bean
Pie
KitKat
Lollipop
Oreo
Marshmallow
Nougat

DESENVOLVIMENTO NO ECOSSISTEMA APPLE

- **Xcode**
  - IDE oficial para desenvolvimento iOS, macOS, watchOS e tvOS.
  - Inclui Interface Builder, simulador, debugger e ferramentas de performance.
- **Swift e SwiftUI**
  - **Swift**: Linguagem oficial de programação para aplicativos Apple.
  - **SwiftUI**: Framework declarativo para construção de interfaces.
- **Frameworks Principais**
  - **UIKit**: Framework tradicional para construção de interfaces gráficas no iOS.
  - **Core Data**: Gerenciamento de banco de dados local.
  - **Combine**: Programação reativa para lidar com eventos assíncronos.
  - **ARKit e RealityKit**: Desenvolvimento de apps com realidade aumentada.
- **Distribuição na App Store**
  - Aplicativos passam por uma revisão antes de serem publicados.
  - Importância das guidelines da Apple (design, segurança, privacidade).
  - Modelos de monetização: compra única, assinaturas, anúncios.

APP STORE: O MARKETPLACE DE APPS DA APPLE

A **App Store** funciona como um marketplace onde usuários podem baixar e comprar aplicativos. O processo envolve:

- ✅ **Publicação de apps**: Desenvolvedores criam e enviam aplicativos através do **Apple Developer Program**.
- ✅ **Revisão da Apple**: Todos os apps passam por um rigoroso processo de **análise e aprovação** para garantir segurança e qualidade.
- ✅ **Distribuição e monetização**: Os apps aprovados ficam disponíveis para download gratuito ou pago.

📌 **Faturamento Total (2023):** Mais de **US$ 85 bilhões** gerados pelos desenvolvedores.
📌 **Quantidade de Apps:** Aproximadamente **2 milhões** disponíveis.
📌 **App mais baixado:** TikTok lidera globalmente.
📌 **Maior mercado:** Estados Unidos, China e Japão.

MODELOS DE MONETIZAÇÃO 💰

- 📌 **1. Apps Pagos**
  - O usuário paga um valor fixo para baixar o app.
- 📌 **2. Compras dentro do App (In-App Purchases - IAP)**
  - Recursos extras ou assinaturas são vendidos dentro do aplicativo.
- 📌 **3. Assinaturas (Subscription Model)**
  - Pagamento recorrente mensal ou anual.
- 📌 **4. Publicidade (Ads)**
  - Exibição de anúncios para gerar receita.

TAXA DA APPLE NA APP STORE

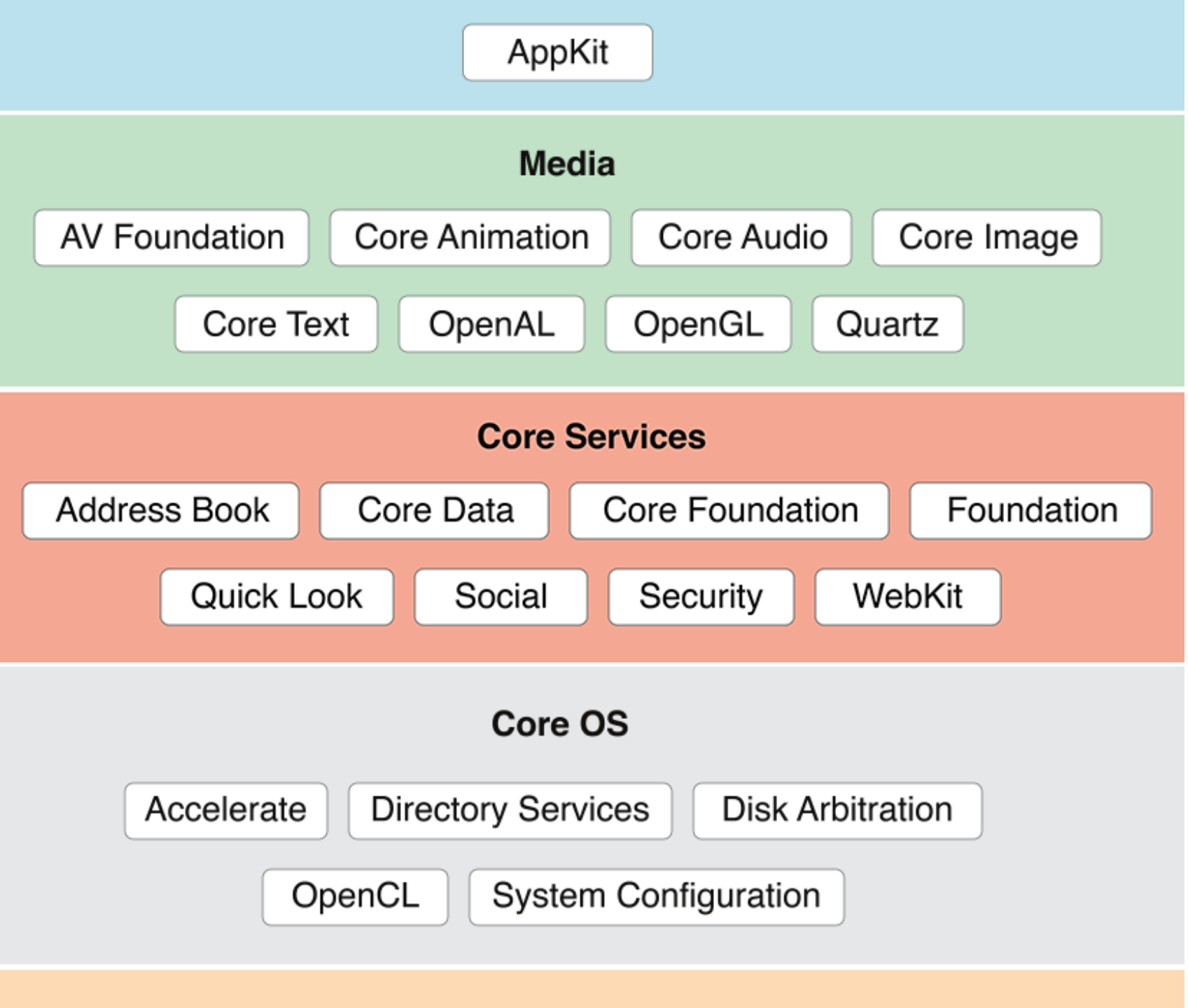
- A Apple cobra uma comissão sobre os aplicativos pagos e compras dentro do app. As taxas padrão são:
  - 30% sobre vendas de apps pagos e compras dentro do app.
  - 15% para desenvolvedores que fazem parte do App Store Small Business Program (ganham menos de US$ 1 milhão por ano).
  - 15% para assinaturas que forem mantidas por mais de 12 meses.
- 💡 Exceções:
  - Empresas como Netflix, Spotify e Epic Games tentam evitar essas taxas, redirecionando usuários para pagamentos externos.
- Regulamentações recentes na União Europeia e em alguns países forçaram a Apple a permitir métodos de pagamento alternativos em alguns casos.

REQUISITOS PARA PUBLICAR UM APP NA APP STORE

1. Criar uma conta no **Apple Developer Program** (custa **US$ 99/ano**).
2. Desenvolver o app usando **Swift e Xcode**.
3. Testar no **TestFlight** antes do lançamento.
4. Enviar para revisão da Apple.
5. Após aprovação, o app fica disponível para download.

PARTICIPAÇÃO DA APPLE NO MERCADO DE DISTRIBUIÇÃO DE APPS

- A App Store é a **segunda maior loja de aplicativos** do mundo, atrás da Google Play Store.
- Em receita, a App Store gera **mais faturamento** do que a Google Play, mesmo com menos downloads.
- No mercado global de apps, a Apple responde por **60 a 70% da receita total**, apesar de ter menos usuários do que o Android.



IOS ARQUITETURA

CAMADAS DA ARQUITETURA DO IOS

- 📌 **1. Core OS (Kernel e Hardware)** 🛠️
  - A base do sistema, responsável pelo **gerenciamento de hardware, segurança, drivers e comunicação com o processador.**
  - ✅ **Baseado no XNU Kernel (Darwin)** – o mesmo do macOS.
    ✅ **Gerencia memória, processos e energia** para otimizar desempenho.
    ✅ **Controle de segurança** com sandboxing, encriptação e permissões de apps.
    ✅ **Frameworks principais:**
  - **Security Framework** → Proteção de dados e autenticação.
  - **Core Bluetooth** → Comunicação com dispositivos Bluetooth.
  - **Accelerate** → Otimizações para cálculos matemáticos e gráficos.

CAMADAS DA ARQUITETURA DO IOS

- 📌 2. Core Services (Serviços do Sistema) ⚙️
- Fornece funcionalidades essenciais para os apps, como banco de dados, redes, geolocalização e armazenamento.
  - ✅ Frameworks principais:
    - Foundation → Manipulação de strings, datas, coleções e APIs.
    - Core Data → Gerenciamento de banco de dados local.
    - CloudKit → Sincronização de dados na nuvem da Apple.
    - Core Location → Acesso à geolocalização.
    - HealthKit → Integração com dados de saúde do usuário.

CAMADAS DA ARQUITETURA DO IOS

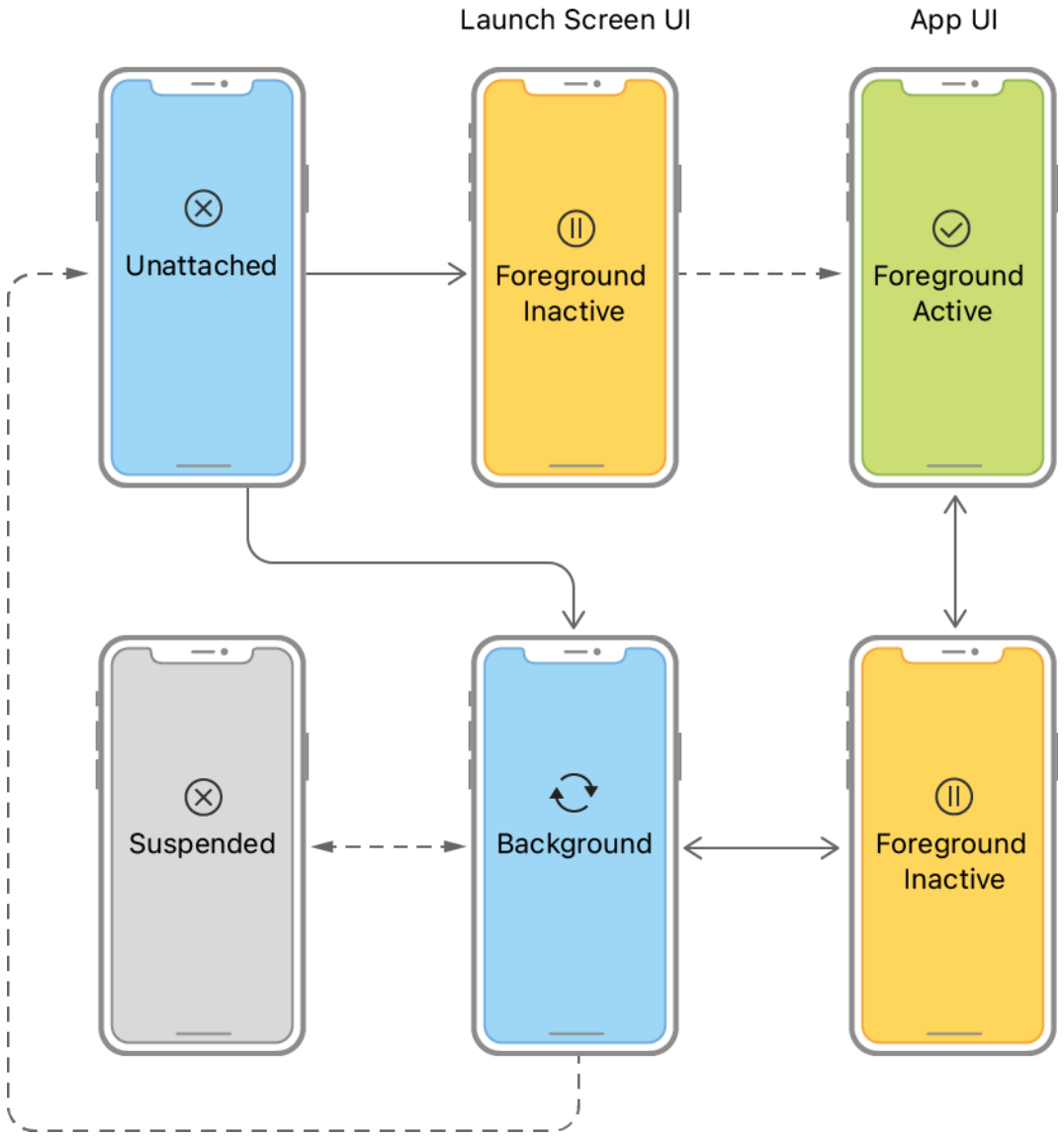
- 📌 **3. Media (Gráficos e Áudio)** 🎨🎶
- **Responsável por gráficos, animações, áudio e vídeos dentro do iOS.**
  - ✅ **Frameworks principais:**
    - **Core Graphics → Renderização de gráficos 2D.**
    - **Metal → API de alto desempenho para gráficos 3D e jogos.**
    - **AVFoundation → Manipulação de áudio e vídeo.**
    - **ARKit → Realidade aumentada.**

CAMADAS DA ARQUITETURA DO IOS

- 📌 **4. Cocoa Touch (Interface do Usuário)** 🎭
  - A camada mais alta, que fornece tudo o que os desenvolvedores precisam para criar **interfaces e interações do usuário**.
  - ✅ **Frameworks principais:**
    - **UIKit** → Criação de telas, botões, animações e navegação.
    - **SwiftUI** → Framework moderno para interfaces declarativas.
    - **SiriKit** → Integração com a assistente virtual da Apple.
    - **Haptic Engine** → Feedback tátil para toques e interações.

FASES DO CICLO DE VIDA DE UM APP

- 1 **Not Running** → O app está fechado e não está sendo executado.
  2 **Inactive** → O app está aberto, mas não interagindo (ex: tela bloqueada).
  3 **Active** → O app está em uso pelo usuário.
  4 **Background** → O app está em segundo plano, mas ainda executa algumas tarefas.
  5 **Suspended** → O app está pausado e pode ser finalizado pelo sistema se precisar liberar memória.



SEGURANÇA NA ARQUITETURA DO IOS 🔒

- A Apple implementa diversas camadas de segurança no iOS, tornando-o um dos sistemas mais seguros do mundo.
- ✅ **Sandboxing** → Cada app roda isolado para evitar acessos indevidos a dados de outros apps.
  ✅ **Face ID / Touch ID** → Autenticação biométrica integrada ao sistema.
  ✅ **App Store Review** → Todos os apps passam por uma verificação antes de serem publicados.
  ✅ **Proteção de Dados** → Arquivos do usuário são criptografados automaticamente.

Swift

O QUE É SWIFT?

Swift é uma **linguagem de programação moderna, rápida e segura**, criada pela Apple para o desenvolvimento de aplicativos no ecossistema Apple (iOS, macOS, watchOS, tvOS e visionOS).

Principais características:

- ✅ **Sintaxe simples e expressiva** – Fácil de aprender e escrever.
- ✅ **Tipagem forte e segura** – Evita erros comuns, como acesso a variáveis nulas.
- ✅ **Alto desempenho** – Compilada e otimizada para rodar rápido.
- ✅ **Interoperabilidade com Objective-C** – Permite usar código legado em projetos antigos.
- ✅ **Orientada a objetos e funcional** – Suporta POO e conceitos da programação funcional.

HISTÓRIA E EVOLUÇÃO DA LINGUAGEM

📌 **2010-2014: Criação e Lançamento**

Swift foi desenvolvido pela Apple sob a liderança de **Chris Lattner**.

Criada para substituir o **Objective-C**, que era usado há décadas no desenvolvimento Apple.

Lançada oficialmente na **WWDC 2014**.

📌 **2015: Código Aberto (Open Source)**

A partir da versão **Swift 2.0**, a Apple tornou a linguagem **open source**, permitindo que fosse usada fora do ecossistema Apple.

Surgiram implementações para **Linux** e suporte experimental para Windows.

📌 **2019-2020: SwiftUI e Maturidade**

Introdução do **SwiftUI**, um novo framework para interfaces gráficas, tornando o desenvolvimento mais intuitivo.

Swift se consolidou como uma das linguagens mais populares para apps móveis.

📌 **2024+: Futuro e Evolução**

Swift continua evoluindo, com melhorias de **performance**, novas funcionalidades e refinamento da sintaxe.

Expansão para mais plataformas, como servidores e machine learning.

COMPARAÇÃO COM OUTRAS LINGUAGENS

Característica	**Swift**	**Python**	**JavaScript**	**Java**	**C++**
Tipagem	Estática e segura	Dinâmica	Dinâmica	Estática	Estática
Facilidade de Aprendizado	Alta	Alta	Alta	Média	Baixa
Desempenho	Alto (compilada)	Baixo (interpretada)	Médio (JIT)	Médio (JVM)	Muito Alto (compilada)
Paradigma	OO + Funcional	OO + Funcional	Funcional + OO	OO	OO
Uso Principal	Apps Apple	Data Science, Web	Web, Frontend	Android, Enterprise	Jogos, Computação de Alto Desempenho
Gerenciamento de Memória	Automático (ARC)	Automático (GC)	Automático	Automático (GC)	Manual

BENEFÍCIOS DO SWIFT

```swift
let numeros = [1, 2, 3, 4, 5, 6]
let pares = numeros.filter { $0 % 2 == 0 }
print(pares) // [2, 4, 6]
```

- ✅ **Segurança Aprimorada**
- Evita **erros comuns**, como acesso a variáveis nulas (optionals).
- Tipagem forte evita bugs antes da execução.
- ✅ **Código Moderno e Expressivo**
- Sintaxe mais limpa e intuitiva.
- Exemplo: Criar uma lista de números pares em **Swift** vs **Java**.

```java
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> pares = numeros.stream()
                .filter(n -> n % 2 == 0)
                .collect( Collectors.toList());
System.out.println(pares); // [2, 4, 6]
```

BENEFÍCIOS DO SWIFT

✅ **Multiplataforma e Expansão**

Além de iOS/macOS, pode ser usado para **desenvolvimento backend (Vapor), sistemas embarcados** e até **inteligência artificial**

✅ **Alto Desempenho**

Swift é compilado, o que o torna **muito mais rápido** do que linguagens interpretadas (como Python).

Usa otimizações da **LLVM** para gerar código eficiente.

✅ **Melhor Gerenciamento de Memória**

Usa **ARC (Automatic Reference Counting)** para otimizar o uso de memória sem precisar de um garbage collector.

AUTOMATIC REFERENCE COUNTING (ARC)

- O **Automatic Reference Counting (ARC)** é o sistema de gerenciamento de memória do Swift. Ele é projetado para **alocar e desalocar memória automaticamente**, garantindo que o uso de memória seja eficiente sem precisar de um **garbage collector (GC)**, como em Java.
- O ARC acompanha **quantas referências ativas** um objeto tem. Quando ninguém mais o referencia, ele é **automaticamente desalocado** da memória.

```
var pessoa1: Pessoa? = Pessoa(nome: "Alice")
pessoa1 = nil // Aqui o objeto é desalocado automaticamente
```

MOTIVOS PARA A ALTA PERFORMANCE DO SWIFT

✅ **Linguagem Compilada e Otimizada**

Swift usa o **LLVM (Low-Level Virtual Machine)** para compilar código nativo altamente otimizado.

Diferente de linguagens interpretadas (Python, JavaScript), que são mais lentas por rodarem em tempo de execução.

✅ **Gerenciamento de Memória Eficiente (ARC)**

Swift usa **Automatic Reference Counting (ARC)** para gerenciar memória automaticamente, reduzindo consumo de RAM.

Diferente do **Garbage Collector (GC)** do Java, que pode causar pausas no app.

✅ **Tipagem Forte e Estática**

Evita verificações desnecessárias durante a execução, tornando a execução mais rápida.

Linguagens de tipagem dinâmica (como Python e JavaScript) fazem mais verificações, impactando o desempenho.

✅ **Estruturas de Dados e Algoritmos Otimizados**

Swift oferece coleções eficientes, como **Array, Dictionary e Set**, altamente otimizadas para manipulação de dados.

COMPARAÇÃO DE PERFORMANCE COM OUTRAS LINGUAGENS

Operação	**Swift**	**Python**	**Java**	**JavaScript**	**C++**
Loop 1M de operações	🚀 Rápido	🐢 Muito lento	🔥 Médio	🔥 Médio	⚡ Muito rápido
Alocação de memória	Eficiente (ARC)	Ineficiente	Consome mais RAM	Consome mais RAM	Manual
Cálculo matemático pesado	Muito rápido	Lento	Rápido	Médio	Muito rápido
Execução Geral	Alto desempenho	Baixo desempenho	Médio-alto	Médio	Muito alto

USO DE SWIFT

- MacOS: XCode
- Windows e Linux: instalar com https://www.swift.org/install/macos/
- Ou usar o web:
  - **Replit:**
  - **Vantagens**:
    - Uma plataforma completa de desenvolvimento online que suporta várias linguagens, incluindo Swift.
    - Permite criar projetos completos, compartilhar código e colaborar com outros usuários.
    - Oferece um ambiente interativo com um console integrado.
    - **Link**: https://replit.com/languages/swift
  - **OneCompiler:**
  - **Vantagens**:
    - Interface simples e fácil de usar.
    - Compilação rápida e eficiente.
    - Suporte para entrada e saída de dados.
    - **Link**: https://onecompiler.com/swift